

SHIBASTEPS

A Smart Productivity Companion Combining a React Web App and an ESP32 Desktop Assistant

NUS Orbital	Team mimimi	Target LoA: Apollo
-------------	-------------	--------------------

“Goals become progress when actions become repeatable.”

Repository	github.com/yyy3254422785-dev/mimimi-map
Technology	React · Vite · Express · Supabase · ESP32 · REST/JSON
Core focus	Long-term goals → daily actions → physical feedback → rewards
Document scope	Milestones 1–3, complexity, implementation, testing, limitations and setup

Prepared for project documentation and final evaluation · July 2026

Executive Summary

ShibaSteps addresses a common productivity problem: large goals do not tell users what to do today. The system converts long-term goals into small dated tasks, supports daily completion and check-in, and reinforces progress through Bone Points, streaks, community sharing and a physical ESP32 desktop assistant. The web application handles planning and social interaction; the Express backend validates and synchronizes state; Supabase provides authentication and persistence; and the device provides an OLED task view, infrared controls, a Pomodoro timer, a completion button and motor-driven physical feedback.

The project targets the Apollo Level of Achievement because its main difficulty lies in integration rather than isolated CRUD screens. A single task may be created in the web planner, selected by the backend, displayed on the OLED, completed through a physical button, synchronized back to the web app, reflected in goal progress and rewards, and protected against duplicate or stale updates. This README documents the motivation, user stories, system design, milestone progress, implementation decisions, engineering tradeoffs, testing evidence and future work required by the project guidance.

Key outcome

The tested core flow works: across 20 participants and 180 task attempts, the overall completion rate was 81.1% (146/180), with an average task score of 7.0/10. The strongest task was completing a task (100% completion, 8.4/10); the clearest improvement area was explaining streaks, Bone Points and check-in conditions.

Contents

1. Project Overview
 2. Target Level of Achievement and Apollo Justification
 3. Milestone 1 — Ideation
 4. Milestone 2 — Prototype
 5. Milestone 3 — Extension and Refinement
 6. Feature Documentation
 7. System Architecture and Data Flow
 8. ESP32 Desktop Assistant
 9. Engineering Complexity, Problems and Tradeoffs
 10. Testing and Evaluation
 11. User Testing Results
 12. Installation and Setup
 13. Security and Privacy
 14. Known Limitations and Future Improvements
 15. Repository and Deliverable Checklist
- [Appendix A. Key Firmware Reference](#)
- [Appendix B. Full Poster and Evaluation Guidance](#)

1. Project Overview

1.1 Problem and Motivation

Traditional to-do lists are effective at recording small actions, but they often fail to bridge the gap between a long-term intention and the next concrete step. Users can create a goal such as “prepare for an examination” or “finish a project”, yet still face uncertainty about what to do today. Repetitive list interfaces also provide limited emotional or physical feedback, which can make sustained use difficult.

ShibaSteps reframes productivity as a repeatable action loop. Users define long-term goals, decompose them into small subtasks, schedule those subtasks as daily tasks, complete them on the web or through a desktop device, receive visible feedback, and share progress. The Shiba Inu theme turns an otherwise utilitarian planner into a companion-like experience.

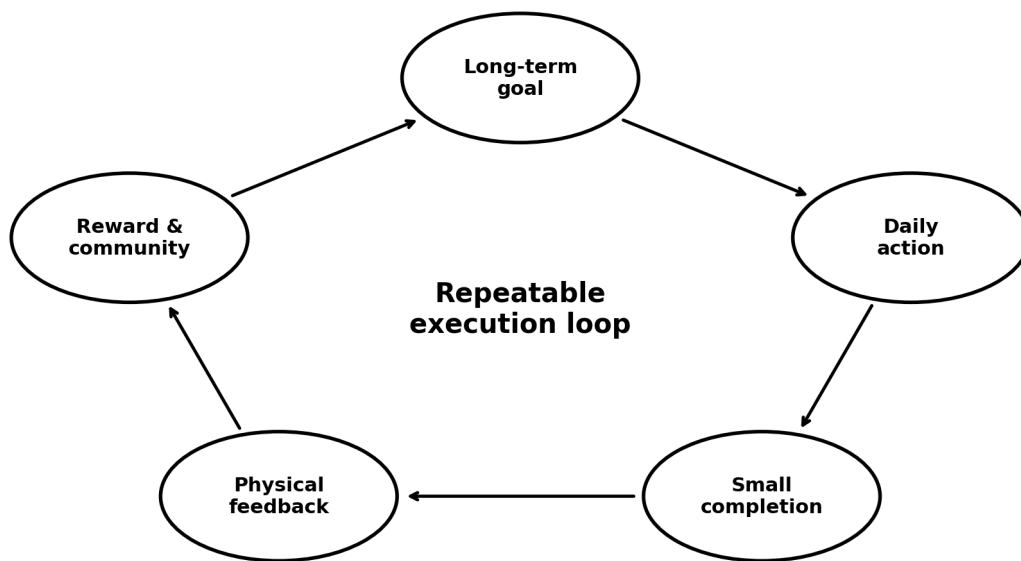


Figure 1. ShibaSteps repeatable execution loop.

1.2 Product Vision

Vision statement

Build a productivity companion that makes large goals actionable, daily progress visible and task completion tangible through one connected software-and-hardware experience.

The project is designed around four product principles:

1. Actionability — every long-term goal should be convertible into a small task that can be scheduled on a real date.
2. Consistency — daily check-in, streaks and carry-over behaviour should encourage return without hiding unfinished work.
3. Tangible feedback — task completion should create a visible response on the OLED and a physical response from the motor.
4. Continuity across layers — web, backend, database and device should represent the same current task, completion state and timer state.

1.3 Core User Journey

1. Create an account or log in.

2. Create one or more long-term goals.
3. Add small subtasks under a selected goal.
4. Assign a subtask to a date so that it becomes a daily task.
5. Select and complete tasks through the web app or ESP32 assistant.
6. Use the shared Pomodoro timer to focus on the current task.
7. Complete the daily check-in, update streak and rewards, and lock the day.
8. Share progress through Dog Circle and return to the next action.

2. Target Level of Achievement and Apollo Justification

2.1 Targeted Level of Achievement

Target LoA: Apollo

ShibaSteps targets Apollo because it combines several non-trivial features into a coherent full-stack and physical-computing system. The core challenge is maintaining correct, secure and understandable behaviour when the same state can be changed through multiple interfaces.

The Apollo guidance asks teams to explain why complex features are complex, what problems made implementation difficult, and which design tradeoffs were made. The following matrix provides that justification.

Complex feature	Why it is complex	Problems encountered	Design decision / tradeoff
Cross-layer task synchronization	The same task exists in the planner, backend state, database and ESP32 memory. Web and device actions may arrive at different times.	Polling intervals, stale state, failed PATCH requests, invalid current-task selection after completion.	Use a server-owned API contract, preserve local completion while retrying, select the next unfinished task, and accept eventual rather than instant consistency.
Goal-to-daily planning model	A subtask belongs to a long-term goal but may also be scheduled on a date, moved, carried over or locked after check-in.	Linking goal progress to dated tasks without duplicating or losing state.	Keep goal identity on tasks and prioritize a clear daily workflow over a highly normalized but harder-to-debug client model.
Hardware interaction state machine	The device must coordinate IR commands, a physical button, OLED pages, timer states, motor motion and network calls.	Button bounce, repeated IR frames, accidental double confirmation and actions during motor movement.	Use explicit timer/screen states, debounce, a two-stage return-button arm, and lock task refresh during confirmation/celebration.
Authentication and private state	The product contains accounts, goals, tasks, posts and device access.	A high-privilege database key must never be exposed to the browser or device.	Use Supabase Auth for users, keep privileged access on the backend, and authenticate the ESP32 with a separate device token.
Gamification and daily check-in	Points and streaks must reflect genuine progress, remain understandable and resist duplicate rewards.	Users were unsure about check-in requirements and the difference between streak and Bone Points.	Enforce eligibility through a locked day and one-time goal-reward state; improve visual guidance based on user testing.
Community layer	Dog Circle adds posts and likes while the rest of the system remains personal by default.	Balancing social motivation with privacy and scope.	Use lightweight progress sharing instead of building a full social network; retain private planning data.

2.2 Evidence Supporting Apollo Scope

1. A deployed React + Vite web application with authenticated user flows and multiple interacting feature areas.

2. An Express backend that mediates web, Supabase and device requests instead of exposing privileged database access.
3. A physical ESP32 prototype integrating OLED, IR remote, button input and a 28BYJ-48 stepper motor.
4. Bidirectional synchronization: tasks are downloaded to the device and completion/current-task/timer updates are sent back.
5. Failure-aware device logic that keeps locally completed tasks completed while network synchronization is retried.
6. A structured user test with 20 unique participants, nine tasks each and task-level completion, scores and issues.
7. Explicit documentation of limitations, tradeoffs and improvements rather than presenting the prototype as a finished commercial device.

2.3 Quantitative Complexity Evidence

Apollo complexity is not inferred from the project's total line count alone. The measurable evidence below identifies the number of interfaces, states, data boundaries and evaluation attempts that the implementation must coordinate.

Complexity dimension	Quantitative evidence	Why it creates engineering work
Execution environments	4: React/Vite, Express, Supabase and ESP32.	One task can move from browser planning to backend persistence and physical-device action; the design accepts eventual consistency.
API surface	17 application/device endpoints: 13 protected /api and 4 /device-api.	The backend validates state, task, timer, profile and community actions for two authenticated clients.
Authentication boundaries	2 mechanisms: Supabase user JWT and fixed ESP32 token.	Private user data and the fixed physical device require separate trust boundaries.
Device state space	4 timer states and 4 screen modes, plus IR, button, motor and Wi-Fi retry behaviour.	The firmware must prevent invalid actions during confirmation and recover after request failure.
Private planner model	6 persisted fields: goals, active goal ID, tasks by date, Bone Points, checked-in dates and carry-over dismissal date.	Goal subtasks, dated task copies and a locked check-in day must remain consistent for one user.
Implementation scale	2,210 lines in App.jsx and 1,295 in server.cjs in the submitted snapshot.	This provides scope context; the interface and state measures above are the primary complexity evidence.
Evaluation coverage	20 participants × 9 scenarios = 180 task attempts.	Testing covers planning, check-in, reward visibility, goal switching and community discovery; this measures evaluation breadth.

3. Milestone 1 — Ideation

3.1 Proposed Problem Statement

People frequently create ambitious goals but fail to translate them into repeatable daily actions. Existing planners may capture deadlines and checklists, but they do not necessarily create a motivating loop between planning,

focused execution, feedback and reflection. ShibaSteps proposes an integrated solution in which software organizes the plan and a desktop assistant makes the current action physically present.

3.2 Primary User Stories

Persona	User story
Goal owner	As a user, I want to create multiple long-term goals so that I can organize different areas of my life.
Planner	As a user, I want to break a goal into small subtasks and schedule them on specific dates.
Daily executor	As a user, I want to see only the tasks relevant to the selected day and mark them complete.
Focus user	As a user, I want a Pomodoro timer tied to my current task on both the web and device.
Hardware user	As a user, I want to choose tasks and confirm completion without reopening the browser.
Returning user	As a user, I want unfinished work to be carried forward and completed days to remain trustworthy.
Motivated user	As a user, I want points, streaks and visible progress so that small actions feel rewarding.
Community participant	As a user, I want to share selected progress and react to others without exposing my full private plan.
Account holder	As a user, I want my goals, tasks and profile to remain separate from other users.

3.3 Requirements and Traceability

The initial user stories were converted into the following verifiable requirements. The acceptance criteria state the observable outcome required of the deployed prototype; the traceability map records the implementation and evaluation evidence available in this submission.

Functional requirements

ID	Requirement	Acceptance criterion	Priority
FR-01	A signed-in user can create multiple long-term goals and add subtasks.	A new goal and its subtasks remain available after refresh and sign-in again.	Must
FR-02	A user can create, complete, delete or postpone dated tasks.	A postponed task is removed from its original date and appears once as incomplete on the next date.	Must
FR-03	A user can complete one daily check-in only after satisfying the daily rules.	Check-in is rejected for a non-current day, a repeated day, an empty list or an unfinished list; success locks the day.	Should
FR-04	User planner data is separated by authenticated account.	A request without a valid user token is rejected, and one account cannot retrieve another account's private state.	Must
FR-05	A user can post, like/unlike and delete their own Dog Circle content.	Empty or >500-character posts are rejected; repeated likes do not create duplicates; non-owners cannot delete a post.	Apollo extension
FR-06	The configured ESP32 can display shared tasks and synchronise task/timer actions.	The device can read state and submit a valid completion or timer update through its token-protected routes.	Apollo extension

Quality requirements

ID	Requirement	Acceptance criterion	Priority
NFR-01	The system validates high-risk API input and preserves a clear state after failures.	Invalid task/timer input is rejected; the device has retry/preservation handling for a failed completion update.	Must
NFR-02	Core workflows are understandable to target users.	Task-based evaluation records completion, a 1–10 score and issue feedback for each of nine scenarios.	Should

3.4 Requirements Traceability Map

This map distinguishes implementation evidence from evaluation evidence. Where no automated test artifact exists, the README identifies the available manual, hardware or user-testing evidence instead of implying automated coverage.

Requirement	Implementation evidence	Evaluation evidence
FR-01	Goal and subtask state in App.jsx; debounced appData persistence through PATCH /api/state.	Functional workflow evidence and participant task 1: Create a goal.
FR-02	Date-keyed tasks plus add, toggle, delete and move-to-tomorrow handlers in App.jsx.	Functional workflow evidence; participant tasks include adding tasks, completion and moving a task to tomorrow.
FR-03	completeDailyCheckIn checks current date, duplicate check-in, non-empty list and all-done condition; checked-in dates lock the UI.	Participant task 5: Try check-in; results identify check-in guidance as a priority iteration area.
FR-04	AuthGate session handling; Bearer token attachment in shibaApi.js; requireAuth on /api routes.	Manual multi-account verification recorded as integration testing; no automated authorization suite is claimed.
FR-05	POST/GET/DELETE post routes and POST/DELETE like routes; server derives canDelete from authenticated owner.	Participant task 8: Browse Dog Circle; community workflow included in manual functional testing.
FR-06	Four token-protected /device-api routes for state, current task, task completion and timer updates.	Live device demonstration, hardware bench testing and failure-path testing.
NFR-01	Route validation for task IDs, boolean completion, timer bounds/status and post content; firmware retry/preservation logic.	Failure-path testing covers disconnected Wi-Fi, failed completion PATCH, stale state and no unfinished task.
NFR-02	Planner, progress and community interface implemented in the deployed web application.	20 participants × 9 scenarios = 180 attempts; completion, score and issue data are analysed in Section 11.

3.5 Evidence Across Development Stages

Stage	Engineering artifact	How it informs the next stage
Milestone 1 — Ideation	Problem statement, user stories, prioritised feature plan, requirements/acceptance criteria and initial architecture split.	Defines what must be built and the boundaries that later implementation must respect.
Milestone 2 — Prototype	Working web/device prototype, layer-by-layer implementation outcomes, prototype screenshots and a problem-resolution record.	Reveals state-management, deployment and input-reliability issues that drive refinement.

Milestone 3 — Extension and refinement

Authentication, user-scoped data, Dog Circle, failure handling, user-testing dataset and prioritised iteration plan.

Uses observed usability and reliability evidence to distinguish completed scope from future improvements.

3.6 Initial Feature Plan

Priority	Planned scope
Must have	Authentication; create/edit goals; add daily tasks; date planner; complete/delete/postpone; persistence.
Should have	Daily check-in; streak; Bone Points; goal progress; carry-over prompt; Pomodoro timer.
Apollo extension	ESP32 synchronization; OLED task display; IR input; physical completion feedback; device authentication; Dog Circle.
Future	Weekly/monthly analytics; mobile-first layout; richer animations; broader hardware enclosure refinement.

3.7 Initial Design and Architecture Plan

The design deliberately separates responsibilities: React owns the interactive interface; Express owns privileged validation and device routes; Supabase owns authentication and persistent state; and the ESP32 owns local hardware input/output. This separation reduces direct coupling, but introduces synchronization and deployment complexity that became a central engineering focus in later milestones.

3.8 Milestone 1 Deliverable Plan

Phase	Deliverable
Ideation	Define target user, pain point, product loop, user stories and target Apollo features.
Web prototype	Implement a usable goal and daily-task flow with local or backend persistence.
Connected prototype	Deploy backend/database and connect the first ESP32 task/timer flow.
Extension	Add auth, check-in, gamification, community and robust failure handling.
Evaluation	Demonstration video, poster, user testing, issue prioritization and final README.

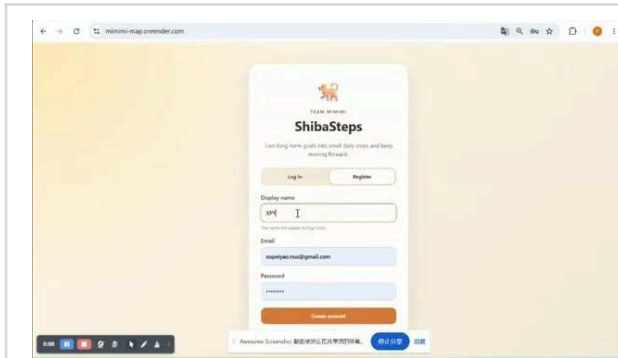
4. Milestone 2 — Prototype

4.1 Core Features Developed

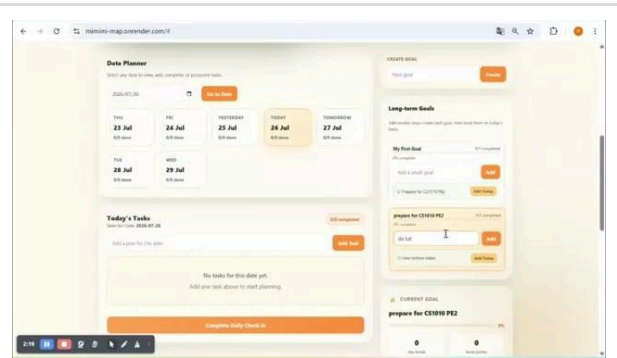
Layer	Prototype outcome
React + Vite frontend	Date planner, long-term goals, daily tasks, completion controls, check-in, progress cards, Pomodoro, profile and Dog Circle.
Express backend	REST API, state validation, device endpoints and protected Supabase access.
Supabase	Authentication, persistence and per-user state.

Layer	Prototype outcome
ESP32 firmware	Wi-Fi, HTTPS/JSON communication, OLED rendering, IR remote mapping, button interrupt, timer state machine and motor control.
Deployment	Cloud-hosted web/backend endpoints configured with environment variables so that the device can reach the system outside local development.

4.2 Prototype Evidence



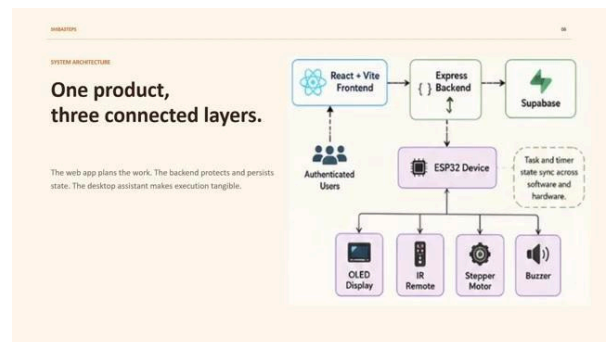
Account registration and authentication flow.



Date planner, goals and daily-task interface.



Physical desktop assistant used beside the web app.



Architecture explanation from the project demo.

4.3 Problems Encountered in the Prototype

Problem	Resolution
Large frontend component and state interactions	Adding carry-over, goal subtasks and device state to the same React component caused misplaced returns, nested effects and Vite/ESLint failures. Effects were moved to the top level, derived state was separated, and update guards were introduced.
Deployment and environment configuration	The browser, backend, Supabase and physical device all required different environment variables and origins. Configuration was moved into environment files and Arduino secrets rather than hard-coded into shared source.
Device display constraints	A 128×64 OLED cannot reproduce the web UI. The device uses concise screen modes, cropped/wrapped task text and short status markers.
Physical input reliability	IR repeat frames and button bounce could trigger unintended actions. Repeat commands are ignored, the button uses an interrupt plus debounce, and completion requires explicit confirmation.

Problem	Resolution
State becoming invalid after completion	Once a task is completed, the current index may point to a hidden task. The firmware searches for the next unfinished task and shows an all-complete state when none remain.

5. Milestone 3 — Extension and Refinement

5.1 Extensions Added

1. Long-term goals with smaller subtasks and per-goal progress.
2. Scheduling subtasks into a selected date as daily tasks.
3. Daily check-in, date locking and streak/reward feedback.
4. Carry-over handling for unfinished work from the previous day.
5. Dog Circle posts and likes for lightweight social motivation.
6. Profile and authentication flows with user-scoped state.
7. ESP32 current-task selection, shared timer updates and task completion.
8. Physical completion celebration through a stepper-driven mechanism.
9. Pending completion retry so temporary network failure does not immediately erase local progress.
10. Filtering completed tasks from the device and automatically selecting the next unfinished task.

5.2 Bugs Squashed and Edge Cases Handled

Edge case	Handling
Server still reports a task as incomplete after the device completed it locally	While a completion PATCH is pending, the downloaded remote state is overridden locally for that task so it remains complete.
No valid current task	The firmware searches for the first unfinished task; if none exist, it displays “All tasks completed”.
Task switching during focus session	Switching is locked while the timer is running or while the completion flow is active.
Timer ending does not necessarily mean task completion	The OLED asks “Task completed?”; the physical button confirms YES and the PAUSE remote key chooses NO.
One press accidentally triggers both completion and motor return	The return action is armed only after the user releases the button and presses it again.
Network refresh interrupts confirmation or motor feedback	Periodic fetch is skipped during confirmation, active celebration and (in the submitted firmware) while the timer is running.
Wi-Fi interruption	The main loop attempts reconnection and separately retries pending completion updates every 10 seconds.
Checked-in day should not remain freely editable	The web flow treats check-in as a daily boundary and prevents later changes to that date.

5.3 User Testing Added as Evaluation Evidence

Milestone 3 included structured testing with 20 unique participants. Each participant completed the same nine task scenarios, producing 180 task attempts. The workbook records experience level, task completion, a 1–10 score and a task-specific issue. A separate summary sheet aggregates completion rate, average score, issue reports, the most common issue and priority.

Interpretation

The result is not “everything is perfect”. It is more useful: the main flow is demonstrably usable, while the data identifies concrete clarity problems in daily check-in, statistics, task movement and goal switching.

6. Feature Documentation

Feature	What it does	Why it matters
Date Planner	Select a date; create, complete, delete or postpone tasks; inspect work on future dates.	Makes the daily action explicit and time-bound.
Long-term Goals	Create multiple goals, add subtasks, switch active goal and track progress.	Connects strategic intention to daily work.
Daily Check-in	Complete one check-in per day after satisfying the required conditions; update streak and lock the day.	Creates a trustworthy daily boundary and return loop.
Bone Points and Progress	Reward completed work and surface progress indicators.	Makes small completions visible and motivating.
Pomodoro Timer	Select duration, start, pause, stop and synchronize state between app and device.	Links planning to focused execution.
Dog Circle	Create lightweight progress posts, browse updates and interact through likes.	Adds selective social motivation.
Profile and Authentication	Register, verify email, log in and edit display information.	Separates users and supports persistent private state.
ESP32 Assistant	Show tasks/timer, navigate with IR, confirm completion with a button, and provide motor feedback.	Turns the current task into a tangible desktop presence.

6.1 Date Planner and Task Lifecycle

The Date Planner is the operational centre of ShibaSteps. A user selects a date, then creates a quick task or schedules a subtask that already belongs to a long-term goal. Each dated task stores its own identifier, text, completion state and, where applicable, the originating goal and subtask identifiers. The planner therefore preserves the link between strategic work and the action scheduled for a particular day, while still allowing one-off tasks that do not belong to a goal.

For the selected date, a task can be completed, deleted or moved to the following day. Moving creates a new unfinished instance on the next date and removes the original from the current date, so a postponed task does not appear twice. Once a date has been checked in, its task list becomes read-only: the interface blocks further task creation, completion changes, deletion and postponement. This prevents the user from retroactively changing a completed day.

6.2 Long-Term Goals and Goal-to-Daily Decomposition

Users can create multiple long-term goals, select one as the active goal and add a set of smaller subtasks under each goal. A subtask is not automatically a daily commitment. The user explicitly adds it to today’s planner, and the resulting dated task retains the source goal and subtask IDs. This distinction matters because a long-term goal is a planning structure, whereas the daily list is the commitment used for check-in and device synchronisation.

When a scheduled goal subtask is marked complete, ShibaSteps updates both the daily task and its original goal subtask. Goal progress is calculated from the proportion of completed subtasks. The system also records whether the goal-completion reward has already been claimed, so completing the final subtask awards the bonus only once rather than again after later re-rendering or state updates.

6.3 Daily Check-in, Streak and Carry-over Rules

Daily check-in is deliberately a gated action rather than a simple button. It is available only for the current date, only once per day, and only after the user has added at least one task and either completed every task or moved unfinished work to tomorrow. A successful check-in records the current date in the user's checked-in-date list and locks the day. The current streak is then calculated by walking backwards through consecutive checked-in dates, starting from today when possible and otherwise from yesterday.

At the beginning of a new day, unfinished tasks from yesterday trigger a carry-over prompt unless the user has already checked in or dismissed that prompt. Choosing carry-over moves only unfinished tasks into today as fresh incomplete task instances; completed tasks remain on yesterday. This makes missed work visible without silently treating it as done, while preserving the option to postpone individual tasks before check-in.

6.4 Reward and Progress Feedback

The implemented reward rules are visible in the planner state. Completing a task adds 10 Bone Points, while unchecking the same task removes 10 points, with the total clamped at zero. Completing the last uncompleted subtask of a goal grants a one-time additional 15-point bonus. Completing a valid daily check-in grants 20 points. Level display is derived from Bone Points, and Dog Circle Points combine Bone Points with ten times the current streak.

This design gives immediate feedback at three scales: a task completion is rewarded instantly, a completed goal receives a larger one-time reward, and repeated daily completion contributes to the streak. The README does not claim that these reward calculations are currently backend-centralised: in the supplied implementation, the private planner state is maintained in React and then persisted to the authenticated user state through the Express API.

6.5 Authentication and Per-user Persistence

ShibaSteps uses Supabase email-and-password authentication. A new user provides an email address, password and display name; the display name is limited to 40 characters and is later shown in Dog Circle. The AuthGate component obtains and observes the current Supabase session, shows the login or registration interface when no session exists, and signs the user out through Supabase when requested.

Every browser request to the application API first obtains the current access token and sends it as a Bearer token. The Express server requires a valid token for all /api routes, derives the user identity from that token, and uses it when reading or updating private state, profile data and community actions. Private planner data—goals, selected goal, tasks by date, Bone Points, checked-in dates and carry-over dismissal state—is saved after a 500 ms debounce. This isolates user data across accounts while avoiding one network write for every keystroke or state update.

6.6 Dog Circle Community Interaction

Dog Circle is a lightweight authenticated community feed. Users can publish a non-empty post of up to 500 characters, browse the newest posts first, like or unlike a post, and delete only their own posts. The server returns each post with its author display name, total likes, whether the current user has liked it, and whether the current user may delete it; the browser therefore does not decide ownership from UI state alone.

Likes use the post-and-user pair as a conflict key and are created with duplicate-ignore behaviour. Repeated clicks therefore do not create multiple likes from the same account. The interface also keeps a pending-post set while a like or deletion is in flight, preventing duplicate requests from rapid repeated interaction. After a successful community action, the feed is refreshed from the server so the displayed count and ownership state come from persisted data.

6.7 Web–Device Focus Assistant

The web application publishes the current task list and timer state through the authenticated Express API. The ESP32 desktop assistant accesses separate /device-api routes with a fixed device token. It can retrieve the current task state, change the current task, update a task completion and synchronise Pomodoro timer fields. The device interface is intentionally constrained to the actions that are useful while studying away from the browser: task navigation, timer control, task completion and immediate physical feedback.

On the hardware, the OLED shows task and timer information; the IR remote supplies duration and navigation commands; the completion button confirms a task action; and the motor and buzzer make completion tangible. Device endpoints validate core inputs, including task IDs, boolean completion state, positive selected minutes, non-negative remaining seconds and one of four timer states: ready, running, paused or finished. The detailed physical control and recovery behaviour is documented in Section 8.

7. System Architecture and Data Flow

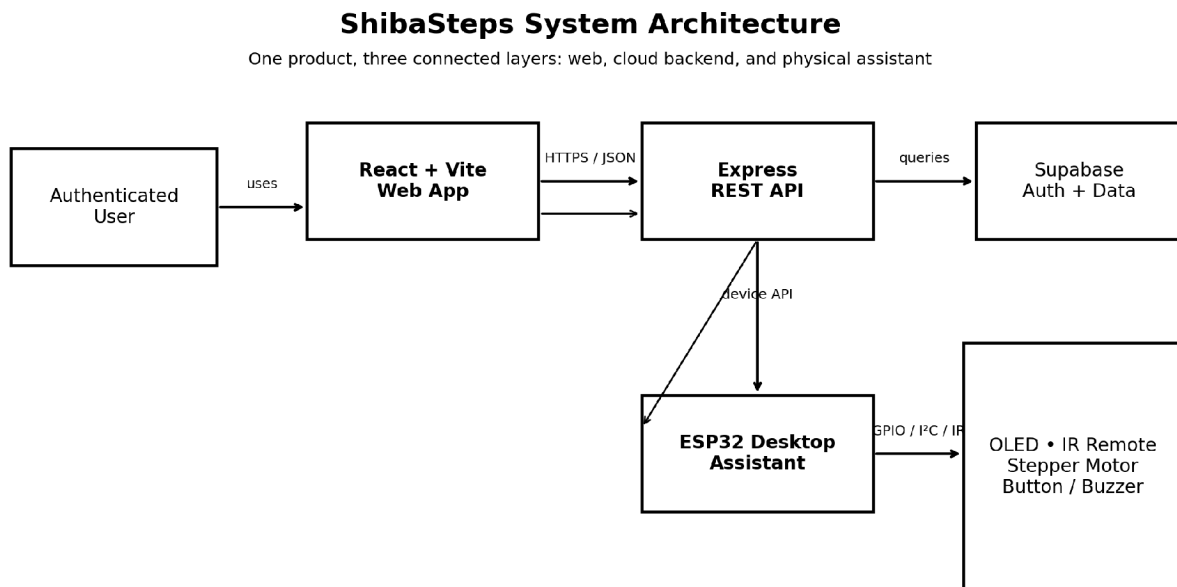


Figure 2. Implemented architecture and communication paths.

7.1 Layer Responsibilities

Layer	Responsibility
React + Vite	Rendering, planner interactions, goal/task editing, check-in, progress, profile, community and browser-side session handling.
Express	REST routes, authentication context, server-side validation, Supabase privileged operations and device API.
Supabase	User authentication and persistent user-scoped application state.
ESP32	Local task cache, current-task selection, Pomodoro state machine, OLED output, IR/button input, motor control and retry logic.

7.2 Device API Contract

Method	Route	Purpose
GET	/device-api/state	Download task list and current task for the device.
PATCH	/device-api/current-task	Update the selected current task.
PATCH	/device-api/tasks/:taskId	Update task completion status.
PATCH	/device-api/timer	Update shared timer state.

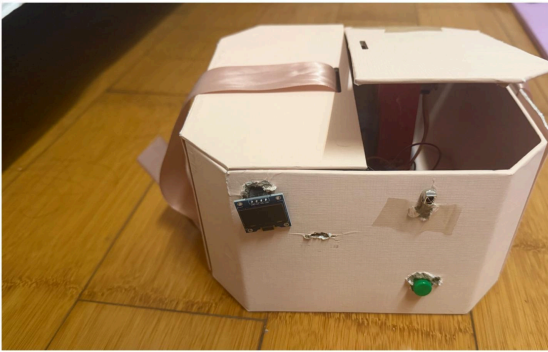
Device requests include a separate Authorization header in the form “Device <token>”. The token is not reproduced in this document. The device uses HTTPS and JSON, with connection and response timeouts to prevent a failed request from blocking indefinitely.

7.3 Synchronization Model

1. The device fetches shared state at startup and periodically when the interaction state is safe to refresh.
2. The current task is sent when the user changes selection on the device.
3. Timer state is sent on key transitions such as setting, starting, pausing, ending and completing.
4. Task completion is optimistic locally: the OLED/motor responds immediately, then the device submits the server update.
5. If the completion request fails, the task is marked as pending and retried; incoming stale state is prevented from reverting it during that retry window.

8. ESP32 Desktop Assistant

8.1 Physical Prototype



Exterior prototype: OLED, IR receiver, green completion button and opening lid.



Interior prototype: ESP32, breadboards, wiring, power bank and motor mechanism.

The enclosure is a functional proof-of-concept rather than a commercial casing. Its purpose is to validate that the task workflow can leave the browser, remain understandable on a constrained display, and produce a physical response when work is completed.

8.2 Hardware Mapping

Component	Pin(s)	Role
IR receiver	GPIO 27	Decode remote commands.
END / confirmation button	GPIO 18	Stop timer, confirm completion and return motor.
OLED SDA / SCL	GPIO 21 / 22	128×64 I ² C display at address 0x3C.
Stepper IN1–IN4	GPIO 25 / 26 / 32 / 33	Drive the 28BYJ-48 motor through an external driver.

8.3 Remote and Button Controls

Input	Action
0–9	Enter a Pomodoro duration, normally 1–99 minutes.
+ / –	Move to the next or previous unfinished task.
START	Start or resume the timer.
PAUSE	Pause the timer; on the completion question, choose NO.
Green END button	Stop timer; choose YES on completion; after release, press again to return the motor.

8.4 Device State Machines

```
enum TimerState : uint8_t {
    TIMER_READY,
    TIMER_RUNNING,
    TIMER_PAUSED,
    TIMER_FINISHED
};

enum ScreenMode : uint8_t {
    SCREEN_MAIN,
    SCREEN_CONFIRM_COMPLETION,
    SCREEN_MESSAGE,
    SCREEN_WELL_DONE
};
```

Simplified excerpt from the submitted ESP32 firmware.

The state-machine approach prevents the same command from having an ambiguous meaning. For example, PAUSE pauses during a focus session but means “No, the task is not complete” on the confirmation screen. The physical button similarly changes role according to state, so button debounce and explicit arming are essential.

8.5 Completion and Failure Flow

1. Timer finishes or the user manually stops it.
2. OLED asks whether the task was completed.
3. YES marks the task complete locally and starts the motor celebration.
4. The device PATCHes the task and timer state to the backend.
5. If synchronization fails, “Saved locally / Sync retrying” is shown and the completion is retried every 10 seconds.
6. After the button is released and pressed again, the motor returns and the next unfinished task is selected.

9. Engineering Complexity, Problems and Tradeoffs

9.1 Polling vs Real-Time Synchronization

A real-time subscription could reduce delay, but it would increase connection management complexity on the ESP32 and make failure recovery harder to explain. The prototype therefore uses explicit updates plus periodic fetching. The tradeoff is eventual consistency: a change may take a short time to appear, which aligns with user-test feedback that moved tasks sometimes took a moment to show.

9.2 Optimistic Device Feedback vs Server Confirmation

Waiting for the server before moving the motor would make successful completion feel slow and would fail completely during a temporary network interruption. The device therefore responds locally first and retries the network update. This improves responsiveness but requires a pending-sync state to stop stale downloads from undoing the local action.

9.3 Constrained Device UI vs Feature Richness

The web app can show calendars, goals, progress and community content, while the OLED has only 128×64 pixels. The device was intentionally limited to the current unfinished task, short status information and the timer. This avoids a miniature copy of the full web interface and keeps the hardware interaction focused.

9.4 Client State vs Backend Access Control

React updates the interface immediately, and the current implementation maintains planner state, day locking and reward calculations in the client before persisting private app data through the authenticated API. The backend currently enforces user authentication and validates task, timer, profile and community route input. A

future hardening step is to centralise irreversible business rules, such as reward issuance and day locking, on the server rather than relying on browser-side state.

9.5 Security vs Prototype Simplicity

The firmware currently calls `client.setInsecure()`, which encrypts traffic but does not validate the server certificate chain. This reduced setup friction for the prototype, but it is not the preferred production design. A deployed version should pin or load the relevant CA certificate, rotate device tokens and persist pending writes across restarts.

9.6 Breadboard Prototype vs Product Enclosure

Breadboards and a handmade enclosure allowed rapid iteration on wiring and motor geometry. The cost is lower robustness, visible wiring and limited portability. The poster therefore presents mobile support and richer device interactions as future work rather than claiming the device is production-ready.

9.7 Design Decision Record

The following decisions make the evaluation criteria explicit. They were selected for a working Orbital prototype with one configured ESP32 assistant, not as claims that the design is the only suitable production architecture.

Decision	Options and criteria	Selected approach and implementation evidence
DR-01: Synchronisation	Options: Supabase Realtime; short REST polling; event-based writes with periodic reads. Criteria: responsiveness, reconnection complexity, implementation risk and request volume.	Selected: REST state API with periodic refresh and PATCH writes. Evidence: the React client refreshes shared state every second, while REST routes remain simple for the ESP32 to call and retry.
DR-02: Device interaction	Options: mirror the complete web UI; restrict the device to focus actions. Criteria: OLED space, physical input count, cognitive load and reliability.	Selected: task navigation, timer control, completion and feedback only. Evidence: the OLED/IR/button/motor combination supports these four actions without text entry or a complex menu.
DR-03: Authentication	Options: expose database access to the browser; backend-mediated access; one shared unauthenticated device route. Criteria: secret protection, user isolation and device setup effort.	Selected: Supabase user JWT for web API requests plus a fixed device token. Evidence: the browser never receives the privileged database key; the device is limited to its route group.
DR-04: Planner representation	Options: fully normalised client entities; date-keyed task lists retaining goal references. Criteria: goal traceability, implementation clarity and support for postpone/carry-over.	Selected: date-keyed tasks with optional goal and subtask identifiers. Evidence: a dated action can move or lock independently while the originating goal still records progress.

10. Testing and Evaluation

10.1 Testing Scope and Evidence Standard

ShibaSteps uses several complementary forms of validation. System testing checks complete user workflows; integration testing checks communication across React, Express, Supabase and the ESP32; negative and failure-path testing checks rejected or recoverable states; hardware bench testing checks physical components; and Section 11 evaluates usability with external participants. Passing one layer does not substitute for another.

Test type	Scope	Evidence	Status
Manual system testing	Authentication, goal/task/date/check-in and Dog Circle workflows.	ST-01 to ST-05; relevant user-testing tasks in Section 11.	Completed manually
Software integration testing	React ↔ Express ↔ Supabase, Supabase Auth and Express ↔ ESP32.	IT-01 to IT-06; milestone verification from 23–25 July 2026.	5 passed manually; 1 partially verified
Hardware bench testing	OLED, IR mapping, button debounce, motor motion and Wi-Fi recovery.	Submitted firmware, serial/API observations and physical demonstration.	Completed manually; no hardware usability study
Negative/failure-path testing	Access rejection, invalid state and network recovery paths.	FT-01 to FT-08.	4 passed manually; 4 code-inspected only
User testing	Nine product tasks across 20 unique participants.	180 attempts with completion, score and issue fields; see Section 11.	Completed
Automated unit/API/UI E2E	Regression coverage for deterministic rules, authorization and browser flows.	No automated test code or CI report was included in the submission.	Not implemented

Evidence status

Passed manually means the team executed the scenario and observed the expected result. Partially verified means the main interface worked but the complete lifecycle was not continuously enabled. Implemented; not separately tested means the guard exists in code but no independent execution record was supplied. Not implemented is reserved for absent automated coverage.

10.2 System and End-to-End Testing

These cases validate complete user-visible outcomes rather than individual functions. The status records functional correctness separately from usability findings.

ID	Scenario and procedure	Expected outcome	Result and evidence
ST-01	Authentication and protected access: register or sign in, open the application, sign out, and sign in again.	A valid session opens the private application; signing out returns to the authentication gate.	Passed manually — Authentication was exercised in the deployed application; account creation/login was also part of prototype verification.
ST-02	Core goal loop: create a goal, add a subtask, schedule it for today, complete it, and inspect goal/reward progress.	The daily task and source subtask become complete, progress updates, and completion feedback is shown.	Passed manually — The team demonstrated the flow; user-testing tasks 1–3 independently exercised goal creation, task entry and completion.
ST-03	Cross-date planning: move an unfinished task to tomorrow, then switch to tomorrow.	The task is removed from today and appears once, incomplete, on tomorrow's list.	Passed manually — The workflow operated correctly; user testing still identified weak confirmation and delayed destination-date visibility.
ST-04	Daily check-in: satisfy the task conditions, check in, then attempt to edit the checked-in day.	Check-in succeeds once, awards the check-in result, and locks the date against later task changes.	Passed manually — The rule worked in team verification; only 14 of 20 users completed the check-in task, showing a usability rather than core-function failure.
ST-05	Dog Circle: create a post, like/unlike it, browse recent posts, and delete an owned post.	The feed refreshes from persisted data, one account contributes at most one like per post, and only the owner can delete their post.	Passed manually — Community workflows were included in functional verification; browsing Dog Circle reached 90% completion in user testing.

10.3 Software Integration Testing

Integration tests target the boundaries where most Apollo-level complexity occurs: authentication, persistence, account isolation and bidirectional software-device state exchange.

ID	Integration path and procedure	Expected outcome	Result and evidence
IT-01	React → Express → Supabase persistence: change goals/tasks, refresh, then sign out and sign in again.	The authenticated account retrieves the same private planner state after a new browser session.	Passed manually — Frontend, backend and Supabase persistence were verified on 23 July 2026; state also remained after backend restart.
IT-02	Supabase Auth → Express authorization: use a valid session to call protected /api routes, then repeat after sign-out.	Valid JWT requests succeed; requests without a session are blocked from private application data.	Passed manually — Login/logout and protected application access were verified in the deployed flow.
IT-03	Per-user isolation: create planner data in Account A, then sign in as Account B.	Account B receives its own state and cannot see Account A's goals or tasks.	Passed manually — Different accounts were verified as independent on 24 July 2026.
IT-04	React → Express → ESP32: publish/select tasks in the web application and allow the device to refresh state.	The device receives a valid task list/current task and displays the selected unfinished task.	Passed manually — ESP32 GET responses and task switching were verified on 25 July 2026.
IT-05	ESP32 → Express → React: complete the current task from the physical device and refresh the web-visible state.	The task-completion PATCH succeeds, the web state reflects completion, and the device advances to an unfinished task.	Passed manually — Task PATCH returned HTTP 200 and the cross-layer completion flow was demonstrated on 25 July 2026.
IT-06	Timer-state integration: submit selectedMinutes, remainingSeconds and status through the timer endpoint.	Valid timer fields are stored and returned consistently to the connected client.	Partially verified — Timer PATCH returned HTTP 200, but periodic timer synchronization is commented out in the submitted main loop and is not claimed as continuously verified.

10.4 Negative and Failure-Path Testing

The table deliberately separates executed recovery tests from guards that were confirmed by code inspection only. This prevents implementation evidence from being overstated as test evidence.

ID	Negative/failure condition	Expected outcome	Result and evidence
FT-01	Call a protected /api route without a valid user access token.	The request is rejected and private state is not returned.	Implemented; not separately tested — requireAuth protects the /api route group, but no independent negative-test log was supplied.
FT-02	Call /device-api with a missing or incorrect device token.	The request is rejected before device state is read or modified.	Implemented; not separately tested — The device route group has a separate token guard; no dedicated rejection record was supplied.
FT-03	Submit an unknown task ID, a non-boolean completion value, or an invalid timer value/status.	The API returns a client error and does not write an invalid state.	Implemented; not separately tested — Validation branches exist for task and timer requests, but no separate execution report was supplied.
FT-04	Attempt check-in on a non-current date, twice on one day, with no tasks, or with unfinished tasks.	The action is blocked with a reason; successful check-in is recorded only after all conditions are met.	Passed manually — Eligibility guards and date locking were exercised; user testing exposed that the rules were not explained clearly enough.
FT-05	Submit an empty or over-500-character post, duplicate a like, or delete a post owned by another account.	Invalid posts/deletion are rejected and repeated likes do not create duplicate records.	Implemented; not separately tested — Server validation and ownership/duplicate constraints exist, but boundary cases were not independently logged.
FT-06	Disconnect Wi-Fi while completing a task locally, then reconnect.	The local completion remains visible, the device reports a pending sync, and the PATCH is retried after connectivity returns.	Passed manually — Wi-Fi interruption and completion retry were part of failure-path verification.
FT-07	Receive stale remote state while a local task-completion PATCH is pending.	The stale download does not revert the locally completed task before synchronization succeeds.	Passed manually — Firmware preserves pending local completion over stale downloaded state.
FT-08	Complete all available tasks or provide no valid current task.	The device selects no hidden/completed task and displays an all-complete state.	Passed manually — The next-unfinished search and no-task state were verified in the firmware flow.

10.5 Hardware Verification Boundary

Hardware bench verification covered OLED initialization, IR command mapping, button debounce, motor forward/return behavior and Wi-Fi reconnection. These checks establish that the prototype components and control paths operate. They do not establish hardware usability: the 20-participant workbook mainly evaluates the web application and does not separately measure OLED readability, remote-control learnability, button behavior or perceived motor feedback. The README therefore treats hardware usability testing as future work rather than merging it with the completed software user test.

10.6 Test-to-Requirement Traceability

This matrix closes the loop from the acceptance criteria in Section 3.3 to executable checks and user evidence. A requirement can be functionally satisfied while still carrying a usability or automation limitation.

Req.	Acceptance area	Linked evidence	Conclusion
FR-01	Create goals and subtasks	ST-02; IT-01; user-testing tasks 1–3	Satisfied for the prototype; task-entry efficiency remains an iteration issue.
FR-02	Create, complete, delete and postpone dated tasks	ST-02–ST-03; IT-01; user-testing tasks 2–5	Functionally satisfied; cross-date confirmation/sync visibility needs improvement.
FR-03	Enforce daily check-in and lock a completed date	ST-04; FT-04; user-testing task 6	Functionally satisfied; only 14/20 users completed the test, so rule explanation remains weak.
FR-04	Isolate private state by authenticated account	ST-01; IT-01–IT-03; FT-01	Satisfied manually; automated authorization regression is absent.
FR-05	Support Dog Circle post/like/ownership rules	ST-05; FT-05; user-testing task 8	Core workflow satisfied; several negative boundaries are code-inspected only.
FR-06	Synchronize the configured ESP32 with task/timer state	IT-04–IT-06; FT-02 and FT-06–FT-08; Section 10.5	Task synchronization satisfied manually; continuous timer synchronization is only partially verified.
NFR-01	Validate risky input and recover from transient failure	FT-01–FT-08	Prototype-level validation/recovery exists; automated, load and security testing are absent.
NFR-02	Provide understandable core workflows	Section 11: 20 participants, 180 attempts	Evidence obtained; check-in, reward labels and movement feedback are prioritized weaknesses.

10.7 Automated Testing Status and Regression Plan

No automated unit, API, component or browser end-to-end suite was included in the submitted artifacts. The following table is a prioritized regression backlog, not a record of executed tests. Automating the P0 items first would protect the rules and authorization boundaries that are easiest to break silently during later changes.

Priority	Layer	Proposed regression coverage	Suggested test setup
P0	Unit logic	Streak calculation; check-in eligibility; task false→true reward transition; one-time goal bonus; move-to-tomorrow transformation.	Proposed: Vitest with pure-function fixtures.
P0	API authorization/validation	Missing/invalid JWT, cross-account state access, device-token rejection, invalid task/timer payloads and post ownership.	Proposed: Supertest with mocked or isolated Supabase data.
P1	Component interaction	Disabled check-in reasons, locked-day controls, pending post actions and visible reward/date-movement feedback.	Proposed: React Testing Library.
P1	Browser E2E	Register/login → create goal → schedule/complete task → check in; postpone task; Dog Circle ownership flow.	Proposed: Playwright against a dedicated test deployment.
P1	Device/API contract	JSON state parsing, task-completion retry, stale-state preservation and timer-state validation without physical hardware.	Proposed: mock HTTP server plus fixed JSON fixtures.

10.8 Engineering Test Limitations

The completed engineering tests are manual and were not captured as a machine-readable CI report with timestamps for every execution. The test matrix improves reproducibility, but another team would still need to rerun the procedures to reproduce the result. The prototype was also verified with one configured ESP32 and limited browser/network combinations; no load, penetration, long-duration reliability or cross-browser compatibility test was supplied.

The most important remaining verification gaps are automated authorization/regression coverage, continuous timer synchronization, a second user-testing round after interface changes, and a dedicated hardware usability study. These limitations do not erase the completed prototype evidence, but they define the boundary of what the current results can support.

11. User Testing Results

11.1 Method and Dataset Structure

The supplied user-testing workbook contains one row for each of 20 unique participants (P01–P20). The participants are grouped by prior familiarity with productivity applications: five were new users, eight were occasional users and seven were frequent users. Each participant attempted the same nine ShibaSteps tasks, producing 180 task attempts in total.

For every task, the workbook records three fields: whether the task was completed, a score from 1 to 10, and a free-text issue comment. It also calculates each participant's overall completion rate, average score and total number of reported issues. This structure allows the evaluation to compare task success, perceived usability and recurring qualitative feedback without presenting survey ratings as a substitute for functional and integration testing.

Scope of the evidence

The workbook supports analysis of task completion, scores, issue comments and experience groups. It does not record participant demographics, session duration, device/browser type, moderation method or long-term retention. This README therefore does not claim findings in those areas.

Dataset item	Recorded value
Participants	20 unique IDs (P01–P20)
Experience mix	5 new · 8 occasional · 7 frequent productivity-app users
Tasks per participant	9
Total task attempts	180
Fields for every task	Completed? · Score (1–10) · Issue comment
Participant-level outputs	Overall completion rate · Average score · Total issues
Evidence source	ShibaSteps_User_Testing_20_Users_Unique_ID(1).xlsx

11.2 Testing Task Script

No.	Task in workbook	Evaluation focus
1	Create a goal	Discoverability of the long-term goal creation entry point.
2	Add three tasks	Efficiency and clarity of repeated subtask entry.
3	Complete one task	Clarity of the core completion interaction and completed state.
4	Move one task to tomorrow	Discoverability and feedback for date reassignment.
5	Switch to tomorrow and view the task	Date navigation, selected-date visibility and synchronization feedback.
6	Try to complete the daily check-in	Understanding of check-in conditions and disabled-state explanations.
7	Check your streak and Bone Points	Findability and distinction between the two gamification concepts.
8	Browse Dog Circle	Comprehension of the community feature and navigation.
9	Switch to another goal	Affordance and active-state feedback of the goal selector.

The sequence covers the main product loop from goal creation and daily planning to completion, rewards, check-in and community browsing. It also includes cross-date and cross-goal actions, which are important because many usability problems occur during transitions rather than on a static screen.

11.3 Overall Findings

81.1% overall completion	146 / 180 successful attempts	7.0 / 10 average score	71 issue reports
-----------------------------	----------------------------------	---------------------------	---------------------

Across all participants and tasks, 146 of 180 attempts were successful, leaving 34 unsuccessful attempts. The overall average score was exactly 7.0/10. The median participant completed 88.9% of the tasks, while individual completion rates ranged from 55.6% to 100%. Participant average scores ranged from 6.11 to 8.11, showing that the prototype was generally usable but not equally clear to every tester.

The failures were not evenly distributed. Daily check-in and the Streak/Bone Points task produced 14 of the 34 unsuccessful attempts, or 41.2% of all failures. Both tasks also received the lowest average scores and the highest number of issue reports. This concentration suggests that the largest usability risk is not the basic ability to create or complete tasks; it is communicating rules, rewards and system status clearly enough for users to understand what happens next.

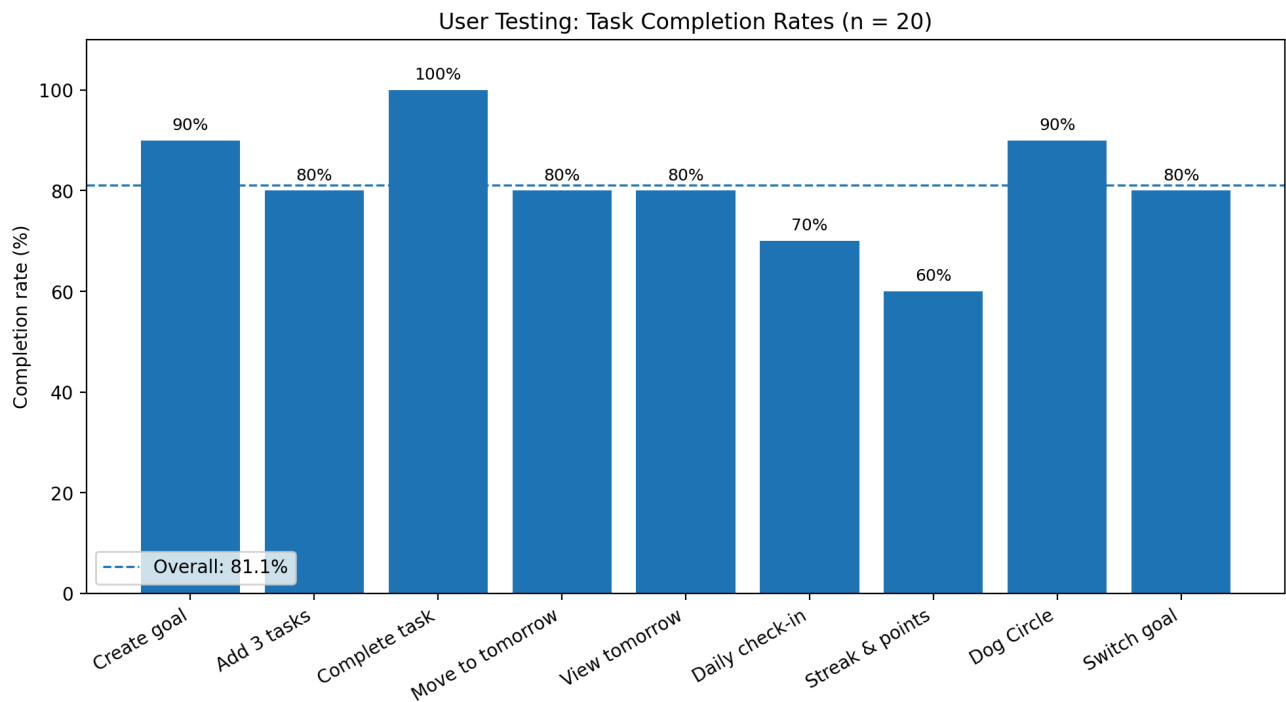


Figure 4. Completion rate by user-testing task.

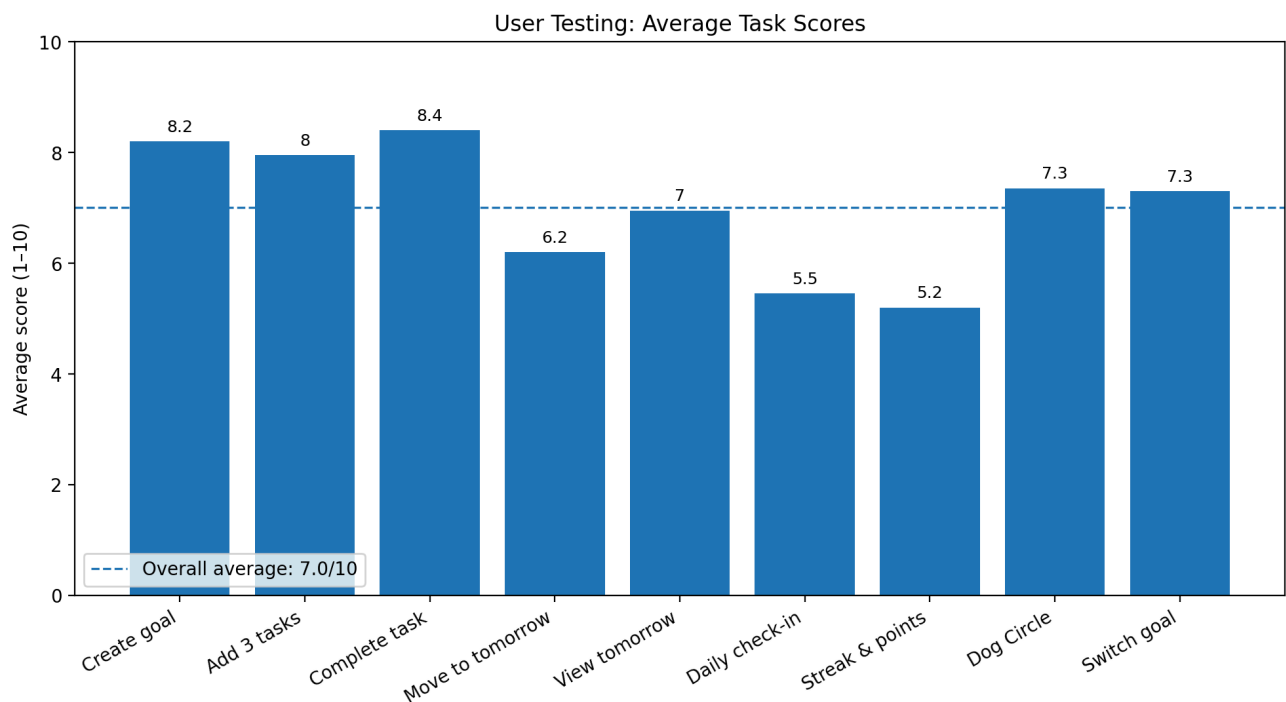


Figure 5. Average score by user-testing task.

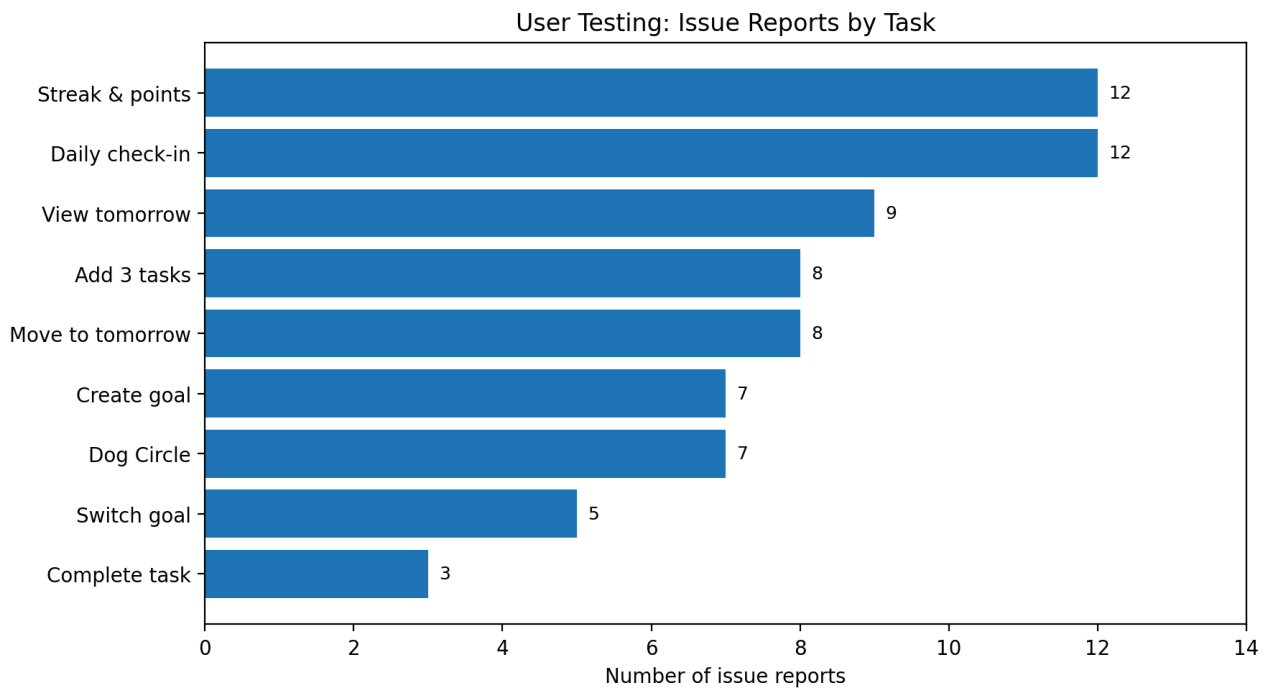


Figure 6. Number of issue reports by task.

11.4 Detailed Task Performance

Task	Successful	Failed	Completion	Average score
Create a goal	18	2	90%	8.20
Add three tasks	16	4	80%	7.95
Complete one task	20	0	100%	8.40
Move one task to tomorrow	16	4	80%	6.20
View the task tomorrow	16	4	80%	6.95
Daily check-in	14	6	70%	5.45
Streak and Bone Points	12	8	60%	5.20
Browse Dog Circle	18	2	90%	7.35
Switch to another goal	16	4	80%	7.30

The strongest result was “Complete one task”: all 20 participants completed it and the task received the highest average score, 8.40/10. This supports the current checkbox-based completion interaction as the most mature part of the web experience. “Create a goal” and “Browse Dog Circle” also reached 90% completion, although their comments show that a successful result does not mean the interface is free from discoverability problems.

The weakest result was “Check your streak and Bone Points”, with 12 successful attempts, eight failures and an average score of 5.20/10. Daily check-in followed with 14 successes, six failures and an average score of 5.45/10. Moving a task to tomorrow achieved a reasonable 80% completion rate, but its score of 6.20/10 shows that users often reached the correct outcome without feeling confident that the action had been registered.

Task	Issue reports	Most common issue	Priority
Create a goal	7	The add-goal button was easy to miss.	Low
Add three tasks	8	Adding several tasks felt repetitive.	Medium
Complete one task	3	The completed state was not obvious enough.	Low
Move one task to tomorrow	8	There was no confirmation that the task moved.	High
View the task tomorrow	9	The moved task took a moment to appear.	Medium
Daily check-in	12	The check-in requirements were not explained.	High
Streak and Bone Points	12	The statistics section needed a clearer label.	High
Browse Dog Circle	7	The Dog Circle label was not immediately understandable.	Medium
Switch to another goal	5	The goal switcher looked like static text.	Medium

The issue-report counts reinforce the score data. Check-in and statistics/rewards each generated 12 comments, while the core completion task generated only three. The date-change workflow produced two related patterns: users wanted an immediate confirmation after moving a task, and several participants reported that the moved task took time to appear on the next date. The design response should therefore combine clearer interaction feedback with more visible synchronization status.

11.5 Participant-Level Results

The following table is a compact participant-level summary taken from the workbook. “Successful tasks” is the number of completed tasks out of nine; “Issues” is the participant’s total number of non-“No issue” comments. Including this table in the README makes the aggregated results traceable to the 20 unique participant records without reproducing all 27 task-specific fields for every person.

ID	Experience	Successful tasks	Completion	Average score	Issues
P01	Occasional	6 / 9	66.7%	6.44	5
P02	Occasional	5 / 9	55.6%	6.33	7
P03	Frequent	9 / 9	100.0%	7.56	1
P04	New	7 / 9	77.8%	6.44	4
P05	Occasional	7 / 9	77.8%	6.67	3
P06	Frequent	6 / 9	66.7%	7.44	4
P07	Frequent	6 / 9	66.7%	7.44	5
P08	Occasional	8 / 9	88.9%	7.56	3
P09	Frequent	9 / 9	100.0%	7.44	5
P10	Occasional	8 / 9	88.9%	6.89	2
P11	New	9 / 9	100.0%	6.44	3
P12	Occasional	8 / 9	88.9%	7.33	3
P13	New	5 / 9	55.6%	6.56	4
P14	New	8 / 9	88.9%	6.11	2
P15	Frequent	5 / 9	55.6%	7.00	5
P16	Frequent	8 / 9	88.9%	7.56	2
P17	Frequent	9 / 9	100.0%	8.11	5
P18	Occasional	8 / 9	88.9%	7.11	3
P19	Occasional	7 / 9	77.8%	6.78	4
P20	New	8 / 9	88.9%	6.78	1

Four participants completed all nine tasks, while three completed five of nine. The highest participant average score was 8.11 and the lowest was 6.11. The number of reported issues ranged from one to seven, with a median of 3.5. These differences show why an overall completion percentage should not be read on its own: a participant may finish most tasks while still reporting several points of hesitation or unclear feedback.

11.6 Frequency of Recurring Issue Comments

The table below lists the most frequently repeated issue statements exactly as recorded in the workbook. Repeated comments are particularly useful for prioritization because they identify problems experienced independently by multiple participants rather than isolated preferences.

Repeated issue statement	Count	Affected area
The check-in requirements were not explained.	7	Daily check-in
Adding several tasks felt repetitive.	5	Task entry
The moved task took a moment to appear.	4	Date navigation / sync
The statistics section needed a clearer label.	4	Rewards and statistics
The next-day arrow was too subtle.	4	Date navigation
The difference between streak and points was not explained.	4	Rewards and statistics
The goal switcher looked like static text.	4	Goal switching
The add-goal button was easy to miss.	3	Goal creation
The streak and Bone Points were difficult to locate.	3	Rewards and statistics
No confirmation appeared after saving the goal.	3	Goal creation feedback
It was unclear whether all daily tasks had to be completed first.	3	Daily check-in
The Dog Circle label was not immediately understandable.	3	Community
The completed state was not obvious enough.	3	Task completion feedback

The most repeated statement concerns unexplained check-in requirements, appearing seven times. When combined with the three comments asking whether all daily tasks must be completed first, the workbook contains ten comments directly related to understanding check-in eligibility. Similarly, reward comprehension is represented by clearer-label, location, icon-meaning and streak-versus-points comments. These patterns justify treating explanation and information hierarchy as core product work rather than cosmetic polish.

11.7 Results by Experience Level

Experience group	n	Attempts	Successes	Completion	Average score	Issues	Issues / user
New to productivity apps	5	45	37	82.2%	6.47 / 10	14	2.80
Occasional users	8	72	57	79.2%	6.89 / 10	30	3.75
Frequent users	7	63	52	82.5%	7.51 / 10	27	3.86

Completion rates were similar for new and frequent users, at 82.2% and 82.5% respectively, while occasional users recorded 79.2%. The more visible difference is in ratings: frequent users averaged 7.51/10, compared with 6.89 for occasional users and 6.47 for new users. This suggests that familiarity with planner conventions improved confidence and perceived ease of use even when users from all groups could often reach the correct result.

New users reported fewer issues per person than the other groups, but also assigned the lowest scores. The workbook does not explain this difference, so it should not be over-interpreted. One plausible interpretation is that new users expressed uncertainty through ratings even when they did not provide a separate free-text issue for every task. This remains an inference rather than a directly measured conclusion.

11.8 Interpretation and Design Implications

1. Preserve the core completion interaction. A 100% completion rate and 8.40/10 score indicate that this flow should be refined rather than redesigned unnecessarily.
2. Make system rules visible before an error occurs. Check-in eligibility should be explained beside the button, with a live count of remaining tasks.
3. Treat reward feedback as an interaction, not only a number. Completion should visibly show “+10 Bone Points”, while a fully completed long-term goal should show its separate bonus.
4. Confirm cross-date actions immediately. After moving a task, show the destination date and whether synchronization has completed.
5. Strengthen affordances. Date arrows, goal switchers, active states and add buttons should visually communicate that they are clickable.
6. Reduce repeated input effort. Adding multiple subtasks should support quick consecutive entry and clearly communicate how Enter/save works.
7. Clarify branded terminology without removing it. “Dog Circle” can retain the ShibaSteps identity while adding a descriptive subtitle such as “Community”.

11.9 Limitations of the User Test

The 20-participant sample is meaningful evidence for a student project, but it is not large enough to establish population-wide usability claims. The workbook does not include age, accessibility needs, prior exposure to ShibaSteps, hardware used, browser, session duration or observation notes. The data also reports one testing round rather than long-term retention, habit formation or repeated daily use.

The tasks mainly evaluate the web application. Although the overall project includes the ESP32 Desktop Assistant, the workbook does not separate web-only results from physical-device interaction results. Hardware usability should therefore be evaluated in a dedicated future test covering OLED readability, remote-control mapping, button behavior, motor feedback, network interruption and recovery.

Scores and issue comments are self-reported and may not capture hesitation, misclicks or time-on-task. A future round should retain the current completion/score/comment structure while adding task duration, observed errors, device type and a short post-test interview. Automated unit and UI end-to-end tests should also be added because user testing cannot reliably detect every regression.

11.10 Prioritized Iteration Plan

Priority	Change	Evidence from testing
High	Explain check-in conditions directly beside the button and show how many tasks remain.	Targets the lowest-scoring workflow and ten directly related eligibility comments.
High	Rename or clarify the statistics area; distinguish Day Streak from Bone Points; animate point gains.	Targets 60% completion, 5.20/10 and repeated reward-comprehension comments.
High	Show an immediate confirmation, destination date and synchronization state after moving a task.	Addresses the 6.20/10 score, move confirmation issues and delayed-appearance comments.
Medium	Make date navigation, goal switching and active states look interactive and selected.	Improves discoverability across date and goal transitions.
Medium	Support rapid multi-subtask entry and clarify Enter-to-save behavior.	Reduces the five repeated “felt repetitive” comments.
Medium	Add a “Community” subtitle to Dog Circle and clearer return navigation.	Preserves branding while improving first-time comprehension.
Medium	Run a second user test after the changes and compare the same nine tasks.	Creates before/after evidence using a consistent task script.

User-testing conclusion

ShibaSteps already supports the core goal-to-task workflow, but the test shows that success alone is not enough. The next milestone should focus on explainability, discoverability and visible feedback—especially for check-in, rewards and date changes—then repeat the same task-based test to measure whether the changes improve both completion and confidence.

12. Installation and Setup

12.1 Prerequisites

1. Node.js and npm.
2. A Supabase project with the required schema initialized from the repository SQL.
3. An ESP32 development board, 128×64 SSD1306 OLED, IR receiver/remote, button, 28BYJ-48 stepper and ULN2003 driver.
4. Arduino IDE or PlatformIO with IRremote, ArduinoJson, Adafruit GFX and Adafruit SSD1306 libraries.
5. A Wi-Fi network that the ESP32 can reach.

12.2 Web Application

```
git clone https://github.com/yyy3254422785-dev/mimimi-map.git
cd mimimi-map
npm install
npm run dev
```

Typical local frontend setup.

The repository root package configuration also includes a backend script:

```
npm run server
# equivalent to: node server/server.cjs
```

12.3 Environment Variables

Location	Variable	Purpose
Frontend	VITE_API_BASE	Base URL of the Express backend.
Backend	SUPABASE_URL	Supabase project URL.
Backend	SUPABASE_SECRET_KEY	Privileged server-side key; never expose to React or ESP32.
Backend	DEVICE_TOKEN	Token accepted by protected device routes.
Backend	PORT	Optional local server port.

12.4 ESP32 Configuration

Create a local `arduino_secrets.h` file. Do not commit real credentials. A safe example is:

```
#pragma once
const char* SHIBA_WIFI_SSID = "YOUR_WIFI_NAME";
const char* SHIBA_WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
const char* SHIBA_API_BASE = "https://YOUR_BACKEND_DOMAIN";
const char* SHIBA_DEVICE_TOKEN = "YOUR_ROTATED_DEVICE_TOKEN";
```

Compile and upload ShibaSteps_ESP32_OLED_Stepper_Hide_Completed.ino. Open the serial monitor at 115200 baud to verify OLED probing, Wi-Fi connection, state download and HTTP status codes.

12.5 Suggested Repository Structure

```
mimimi-map/
├── src/           # React + Vite frontend
├── server/        # Express backend (server.cjs)
├── supabase/      # SQL setup / schema
├── firmware/      # ESP32 sketch and secrets example (recommended)
├── public/        # Static assets
├── package.json
├── package-lock.json
├── render.yaml    # Deployment configuration, if used
├── .env.example
└── README.md / README.docx
```

13. Security and Privacy

1. User authentication is handled through Supabase Auth; application data should be scoped by authenticated user identity.
2. The Supabase privileged key belongs only on the backend. It must not appear in Vite environment variables or ESP32 firmware.
3. The ESP32 uses a separate fixed device token so that hardware access can be revoked independently of database credentials.
4. Task and timer updates should be validated server-side rather than trusted solely because they came from a device.
5. Real Wi-Fi credentials and the submitted device token are intentionally omitted from this README. Any credential exposed in shared files or screenshots should be rotated.
6. The firmware uses `client.setInsecure()` in the submitted prototype; certificate verification is required for a stronger deployment.

Secrets policy

Commit `arduino_secrets.example.h` and `.env.example` only. Add `arduino_secrets.h`, `.env` and `.env.*` to `.gitignore`, while allowing a deliberately sanitized `.env.example`.

14. Known Limitations and Future Improvements

Limitation	Current effect	Improvement
Device task capacity	Firmware stores at most 10 tasks.	Use dynamic allocation carefully or paginate/select only today's most relevant tasks.
Sync latency	Task fetch uses polling; user testing observed delayed appearance.	Use shorter adaptive polling, server push where practical, or explicit refresh feedback.
Timer synchronization	A periodic timer-sync function exists but is commented out in the submitted loop.	Re-enable after measuring network/power cost and server load.
Retry count message	The retry wrappers currently use <code>maxAttempts = 1</code> while the log says three attempts.	Align implementation and message; use bounded exponential backoff.
Pending sync persistence	Pending completion exists only in RAM.	Store pending writes in ESP32 Preferences so a restart does not lose them.

Limitation	Current effect	Improvement
TLS verification	client.setInsecure() skips certificate validation.	Install/pin the correct CA certificate.
Motor blocking	A 1024-step motion with delay blocks input/networking for roughly two seconds.	Use a millis()-driven non-blocking stepper scheduler.
Buzzer mismatch	The poster architecture includes a buzzer, but the supplied firmware focuses on OLED, IR, button and stepper.	Implement the buzzer or label it clearly as planned hardware.
Mobile experience	Current desktop layout is stronger than mobile.	Introduce responsive/mobile-first controls and test on touch devices.
Analytics	Current progress is primarily daily/goal based.	Add weekly/monthly trends, carry-over insight and streak explanations.
Automated tests	No automated suite was supplied.	Add unit, API integration and UI E2E tests to prevent regression.

15. Repository and Deliverable Checklist

Requirement	Evidence in this README	Status
Target LoA stated	Apollo level is stated and justified through six complex feature areas.	Included
Milestone 1	Problem motivation, user stories, design and plan.	Included
Milestone 2	Core features and prototype problems.	Included
Milestone 3	Extensions, bugs/edge cases, problems and user testing.	Included
Complexity explanation	Problems and design tradeoffs for complex features.	Included
User testing evidence	20-participant workbook, task metrics and prioritized findings.	Included
Other testing	Functional, integration, hardware and failure-path testing; automation gap stated.	Included
Code/setup documentation	Web, backend, environment and firmware setup.	Included
Security	User/device separation, secret handling and known TLS limitation.	Included
Visual evidence	Poster, demo frames, hardware photographs and charts.	Included

15.1 Deliverable Files Used to Prepare This README

Artifact	File	Use
Project poster	image.png	Feature overview, architecture, technology and future improvements.
Project video	48371_1785074642.mp4	Web flow, hardware demonstration, architecture and testing narrative.

Artifact	File	Use
ESP32 firmware	ShibaSteps_ESP32_OLED_Stepper_Hide_Completed.ino	Hardware mappings, state machines, device API and failure handling.
User test workbook	ShibaSteps_User_Testing_20_Users_Unique_ID(1).xlsx	Raw participant results and task summary.
Hardware photographs	image(1).png, image(2).png	Exterior and interior physical prototype.
Evaluation guidance	Three supplied screenshots	README milestone requirements, testing guidance and Apollo complexity guidance.

Conclusion

ShibaSteps demonstrates a complete productivity loop rather than a collection of disconnected screens. Long-term goals become dated actions; dated actions become a current task; the current task can be focused on and completed from either the web or a physical assistant; completion produces visual, mechanical and social feedback; and the resulting state returns to the planner. The user test shows that the core flow is usable, while also identifying the precise clarity work needed for the next iteration.

The project's Apollo-level value lies in the engineering across boundaries: browser state, authenticated backend logic, persistent cloud data, constrained embedded UI, physical motion and human-centred evaluation. The prototype is transparent about its limitations, which provides a credible foundation for further testing, mobile refinement, stronger security and more robust reward logic.

Appendix A. Key Firmware Reference

A.1 Core Constants

```
const int IR_RECEIVE_PIN = 27;
const int END_BUTTON_PIN = 18;
const int OLED_SDA_PIN = 21;
const int OLED_SCL_PIN = 22;
const int MOTOR_IN1 = 25;
const int MOTOR_IN2 = 26;
const int MOTOR_IN3 = 32;
const int MOTOR_IN4 = 33;
const int MAX_TASKS = 10;
```

A.2 Pending Completion Preservation

```
if (pendingCompletionSync && remoteId == pendingCompletionTaskId) {
    remoteDone = true;
}
```

When the server has stale data, a locally completed task remains completed while retry is pending.

A.3 Completion Retry

```
if (sendTaskCompletedToServer(pendingCompletionTaskId)) {
    pendingCompletionSync = false;
    pendingCompletionTaskId = "";
    completionSyncFailed = false;
}
```

A.4 Main-Loop Responsibilities

```
process IR input
update button arm and handle button request
update timer and temporary messages
maintain Wi-Fi
retry pending completion
periodically fetch safe task state
render OLED screen
```

The submitted sketch is approximately 2,300 lines including spacing and comments. It combines networking, parsing, UI rendering, input handling, timer logic, motor control and failure recovery in one embedded program. Future refactoring could separate these responsibilities into modules, but the existing explicit functions make the state transitions traceable for a prototype.

Appendix B. Full Poster and Evaluation Guidance

ShibaSteps

A Smart Productivity Companion Combining a React Web App and an ESP32 Desktop Assistant

Orbital Project • Team mimimi

Ready for a walk!

Bone Points
 1,250
 +120 today

Day Streak
 12
 Keep it up!

Progress
 68%
 34 / 50 tasks

Today's Mood
 Pawsitive!

1 Problem & Idea

- Traditional to-do lists feel repetitive and easy to ignore.
- ShibaSteps turns long-term goals into small daily steps.
- The system combines digital planning with a physical desktop companion to make productivity more engaging, visible, and rewarding.

2 Key Features

Date Planner
Create, complete, delete, and postpone tasks by date.
 Long-term Goals
Manage multiple goals, add small subtasks, and track progress.

Daily Check-in
One check-in per day; completed days are recorded and tasks lock after check-in.
 Gamification
Earn Bone Points and build a Day Streak.

Dog Circle
Share short updates, like posts, and interact with other users.
 Profile & Auth
Register, log in, and edit display name.

Pomodoro Timer
Shared timer state between the web app and device.

3 System Architecture

```

graph TD
    Users[Authenticated Users] --> Frontend[React + Vite Frontend]
    Frontend --> Backend[Express Backend]
    Backend --> Supabase[Supabase]
    Backend <--> ESP32[ESP32 Device]
    ESP32 --> OLED[OLED Display]
    ESP32 --> IR[IR Remote]
    ESP32 --> Motor[Stepper Motor]
    ESP32 --> Buzzer[Buzzer]
    ESP32 -.-> Sync[Task and timer state sync across software and hardware.]
  
```

4 How It Works

- User logs in and creates long-term goals.
- Small subtasks are added to a selected day as daily tasks.
- The web app syncs today's tasks to the backend.
- The ESP32 desktop assistant displays tasks and timer information.
- Physical interactions on the device can update completion status.
- Daily check-in updates streak and rewards points.
- Users can share progress in Dog Circle.

5 Implementation Highlights

- Frontend: React, Vite, Motion, CSS
- Backend: Node.js, Express, REST APIs
- Persistence: Supabase authentication + per-user private state
- Community data: posts, likes, profiles
- Device access: token-protected device API
- State synchronization: tasks, current task, timer, completion updates

6 What Makes It Different

- Blends software and hardware into one productivity loop.
- Encourages consistency through visible rewards and check-ins.
- Supports both personal planning and lightweight community interaction.
- Makes task completion tactile through a real desktop assistant.

7 Future Improvements

- Weekly and monthly analytics
- Stronger carry-over and streak insights
- More device animations and richer interactions
- Mobile support

8 Tech Stack

React
 Vite
 JavaScript
 CSS
 Node.js
 Express
 Supabase
 ESP32
 REST API
 JSON

Personalized per-user state
 Web + hardware synchronization
 Gamified study companion
 Community interaction via Dog Circle

B.1 README Requirements Interpreted

The supplied evaluation slide requires the README to state the target level of achievement and document Milestone 1 (ideation), Milestone 2 (prototype), and Milestone 3 (extension), including problems encountered, bugs/edge features and user testing. The Apollo note additionally asks teams to explain why complex features are complex, the implementation problems involved and the design tradeoffs made. These requirements are addressed in Sections 2–5 and 9–11.

B.2 Testing Guidance Interpreted

The mentor guidance recommends a meaningful user-testing sample and survey evidence, while also warning that user testing alone is insufficient. Sections 10–11 therefore separate user testing from functional, integration, hardware and failure-path testing, and explicitly identify automated unit/UI tests as unfinished work rather than inventing results.